

DevOps Laboratory

Experiments 1 – 10

Complete Step-by-Step Reference

Student: Omkar Barad

Tools: Linux | Git | GitHub | Maven | Jenkins | Docker | Selenium | TestNG

Experiment	Title
Exp 1	Linux Commands for DevOps
Exp 2	Staging, Commit and Push on GitHub
Exp 3	Commit, Branches, Forking on GitHub
Exp 4	CI/CD with Jenkins and Java
Exp 5	Docker Commands
Exp 6	Containerization using Docker
Exp 7	Maven Build Lifecycle
Exp 8	Selenium Automated Testing
Exp 9	TestNG Framework Testing
Exp 10	Full DevOps Lifecycle Web Application

Experiment No. 1 — Linux Commands for DevOps

Objective: Implement various Linux commands related to DevOps.

Step 1 — Directory Commands

mkdir — Create directories

```
mkdir mydir          # create single directory
mkdir dir1 dir2       # create multiple directories
mkdir parent/child    # create nested directory
```

ls — List files and folders

```
ls                  # list files in current directory
ls -l              # long format with permissions
ls -a              # show hidden files (starting with .)
ls -r              # show subfolders recursively
ls -la             # combine long + hidden
```

cd — Change directory

```
cd mydir           # go into directory
cd ..              # go back one level
cd ~               # go to home directory
cd /home/ubuntu    # go to absolute path
```

Step 2 — File Commands

touch — Create empty files

```
touch file.txt      # create empty file
touch f1.txt f2.txt # create multiple files
```

echo — Write data into files

```
echo "Hello World" > file.txt # create file with content (overwrites)
echo "New line" >> file.txt    # append to existing file
```

cat — View file content

```
cat file.txt        # display file content
cat >> file.txt      # append content interactively (Ctrl+C to stop)
```

nano — Open a text editor

```
nano file.txt        # open file in nano editor
# Ctrl+O to save, Ctrl+X to exit
```

Step 3 — Copy and Move Commands

cp — Copy files and folders

```
cp file.txt /home/ubuntu/test/          # copy file
cp -r /home/ubuntu/folder1 /home/ubuntu/test/ # copy folder recursively
```

mv — Move (cut/paste) files and folders

```
mv file.txt /home/ubuntu/test/          # move file
mv /home/ubuntu/folder1 /home/ubuntu/test/ # move folder
```

Step 4 — Delete Commands

```
rm file1.txt file2.txt    # remove files
rmdir emptyfolder         # remove empty folder
rm -r folder              # remove folder and all contents
```

Step 5 — Installing Software

```
sudo apt-get update      # update package list
sudo apt-get install apache2 # install apache2
sudo apt-get install git  # install git
sudo apt-get install docker.io # install docker
```

Step 6 — grep — Search in Files

```
grep 'error' logfile.txt      # search for 'error' in file
grep -i 'error' logfile.txt    # case-insensitive search
grep -w 'error' logfile.txt    # search exact word only
grep -iw 'error' logfile.txt   # case-insensitive + exact word
grep -v 'error' logfile.txt    # show lines NOT matching pattern
```

Step 7 — ps — Process Status

```
ps          # show processes for current user
ps -e       # show all background processes
ps -f       # show frontend processes
ps -alx     # show all frontend and backend processes
ps -e | grep 'ssh' # filter processes by name using pipe
```

Step 8 — Pipeline (|) Command

The pipe (|) redirects output of one command as input to the next command.

```
ps -e | grep 'apache' # find apache processes
ls -l | grep '.txt'   # find .txt files in listing
cat file.txt | grep 'keyword' # search keyword inside file
```

Step 9 — awk — Process Tabular Files

```
awk '{print $0}' reports.txt      # print all columns (entire row)
awk '{print $1}' reports.txt     # print first column only
awk '{print $2,$3}' reports.txt  # print 2nd and 3rd columns
awk '{print NR,$2,$3}' reports.txt # add row numbers to output
awk '{print NF}' reports.txt     # print number of fields in each row
awk '{if (NR==2) print $0}' reports.txt # print 2nd row entirely
awk '{if (NR==2) print $4}' reports.txt # print 4th column of 2nd row
```

NF = Number of Fields (columns). NR = Number of Records (rows). \$0 = entire row. \$1,\$2... = specific column.

Step 10 — ping — Test Network Connectivity

```
ping google.com                # ping a server
ping 192.168.1.1               # ping an IP address
ping -c 4 google.com           # send only 4 packets
```

Step 11 — alias — Create Command Shortcuts

```
alias update='sudo apt-get update' # create temporary alias
alias ll='ls -la'                  # shortcut for ls -la

# To make permanent:
sudo su
nano ~/.bashrc
# Add at bottom: alias linux='sudo apt-get update'
source ~/.bashrc                  # reload bashrc
```

Step 12 — man — Manual / Documentation

```
man ls                          # documentation for ls command
man grep                        # documentation for grep
man git                         # documentation for git
```

Command	Purpose
mkdir name	Create a new directory
ls -l	List files with permissions in long format
ls -a	List all files including hidden ones
cd ..	Go back one directory level
touch file.txt	Create an empty file
echo 'text' > file	Create file with content
echo 'text' >> file	Append content to existing file
cat file.txt	Display file content
cp file /path/	Copy file to another location

<code>mv file /path/</code>	Move file to another location
<code>rm file.txt</code>	Remove a file
<code>rm -r folder</code>	Remove folder and all its contents
<code>grep -i 'word' file</code>	Case-insensitive search in file
<code>ps -e</code>	Show all running processes
<code>cmd1 cmd2</code>	Pipe output of cmd1 as input to cmd2
<code>alias x='command'</code>	Create shortcut for a command
<code>man command</code>	Show documentation for a command

Experiment No. 2 — Staging, Commit and Push on GitHub

Objective: Perform version control using Git — staging, committing and pushing to GitHub.

Step 1 — Install Git

Windows:

1. Go to <https://git-scm.com/> and download Git for Windows
2. Run the installer with all default settings
3. Verify:

```
git --version
```

Ubuntu/Linux:

```
sudo apt install git
git --version
```

Step 2 — Configure Git (First Time Only)

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
git config --list      # verify config
```

Step 3 — Create a GitHub Account and Repository

4. Go to <https://github.com> and create an account
5. Click New (top left) → Enter repository name (e.g. MyFirstRepo)
6. Select Public → click Create repository
7. Copy the repository URL shown on the page

Step 4 — Initialize a Local Git Repository

```
mkdir C:\MyProject
cd C:\MyProject
git init          # initialize empty git repository
git status        # check current status
```

Step 5 — Create a File and Add Source Code

Create a file named HelloWorld.java:

```
public class HelloWorld {
    public static void main(String[] args) {
```

```
        System.out.println("Hello from GitHub!");
    }
}
```

Step 6 — Stage the Changes

```
git status          # shows HelloWorld.java as untracked
git add HelloWorld.java  # stage specific file
git add .           # stage ALL files in current directory
git status          # shows file as staged (green)
```

Step 7 — Commit the Changes

```
git commit -m "Added HelloWorld Java file"
git log          # view commit history
git log --oneline # compact view
```

Step 8 — Connect to Remote GitHub Repository

```
git remote add origin https://github.com/YOUR_USERNAME/MyFirstRepo.git
git remote -v          # verify remote is added
```

Step 9 — Push to GitHub

```
git branch -M main          # rename branch to main
git push -u origin main     # push to GitHub
# Enter your GitHub username and Personal Access Token when prompted
```

To get a Personal Access Token: GitHub → Settings → Developer settings → Personal access tokens → Generate new token → check repo scope → copy token → use as password.

Step 10 — Verify on GitHub

8. Go to https://github.com/YOUR_USERNAME/MyFirstRepo
9. You should see HelloWorld.java in the repository
10. Click on the file to view its contents

Step 11 — Pull Changes from GitHub

```
git pull origin main          # pull latest changes from remote
```

Step 12 — Undo Changes

```
git checkout filename.txt     # discard changes in working directory
```

```
git reset HEAD filename.txt      # unstage a staged file
git revert commit-hash           # undo a pushed commit safely
```

Command	What it does
git init	Initialize an empty local Git repository
git status	Show status of working directory and staging area
git add <file>	Stage a specific file for commit
git add .	Stage all changed files
git commit -m 'msg'	Commit staged files with a message
git log	Show full commit history
git log --oneline	Show compact one-line commit history
git remote add origin URL	Connect local repo to remote GitHub repo
git push -u origin main	Push commits to GitHub (first time)
git push	Push commits to GitHub (after first time)
git pull origin main	Download and merge changes from GitHub
git clone URL	Copy a remote repository to local machine
git checkout file.txt	Discard local changes to a file

Experiment No. 3 — Commit, Branches, Forking and GitHub

Objective: Implement Commit, Modification on Branch Master, Forking and GitHub Repository Link.

Step 1 — Clone a Remote Repository

```
git clone https://github.com/YOUR_USERNAME/MyFirstRepo.git # clone to current
folder
git clone https://github.com/YOUR_USERNAME/MyFirstRepo.git C:\gitrepo1 # clone
to specific folder
cd MyFirstRepo
```

Step 2 — View All Branches

```
git branch # show local branches
git branch -a # show all branches (local + remote)
```

Step 3 — Create New Branches

```
git branch mergebranch # create branch named mergebranch
git branch rebasebranch # create branch named rebasebranch
git branch # verify both branches created
```

Step 4 — Switch to New Branch and Add a Commit

```
git checkout mergebranch # switch to mergebranch
echo 'merge file' > merge.txt # create a new file
git add merge.txt
git commit -m "Added merge.txt on mergebranch"
```

Step 5 — Push Branch to Remote Repository

```
git push origin mergebranch # push single branch
git push --all origin # push all branches
git branch -a # verify remote branch exists
```

Step 6 — Merge Branches into Master/Main

```
git checkout main # go to main branch
git pull origin main # pull any remote changes
git branch --merged # see which branches are already merged
git merge mergebranch # merge mergebranch into main
git push origin main # push merged result to GitHub
```

Step 7 — Rebase Branches

```
git checkout main
echo 'rebase file' > rebase.txt
git add rebase.txt
git commit -m "Added rebase.txt"
git rebase rebasebranch      # rebase main on top of rebasebranch
```

Step 8 — Delete Branches

```
git branch --merged          # check merged branches before deleting
git branch -d mergebranch    # delete local branch (safe)
git branch -a                # verify deletion
git push origin --delete mergebranch # delete branch on remote GitHub
```

Step 9 — Undo Staged Changes

```
git checkout filename.txt    # discard changes in working directory (not staged)
git reset HEAD filename.txt   # unstage file (keep changes in working dir)
```

Step 10 — Amend the Last Commit

```
git commit --stat            # see stats of last commit
git commit --amend -m "Updated Message" # change only the commit message
# Then to add more files to last commit:
git add newfile.txt
git commit --amend           # adds newfile.txt to previous
commit
```

Step 11 — Move Commit to a Different Branch (cherry-pick)

```
git log --oneline            # find commit hash you want to move
git checkout newbranch       # switch to destination branch
git cherry-pick abc1234       # copy that commit to this branch
# Then remove commit from original branch:
git checkout main
git reset --soft abc1234      # move commit back to staging area
git reset abc1234             # mixed: move to working directory
git reset --hard abc1234      # hard: discard changes completely
git clean -df                 # remove untracked files and directories
```

Step 12 — Recover Deleted Commits (reflog)

```
git reflog                   # see all past HEAD positions
git checkout abc1234          # go back to that commit
```

```
git branch backup          # create backup branch from that commit
git checkout main          # back to main
git checkout backup        # verify backup branch has the files
```

Step 13 — Revert a Published Commit

```
git revert abc1234          # creates a new commit that undoes abc1234
# A text editor opens — save and close it
git diff abc1234 HEAD      # compare original and reverted
```

Step 14 — Stash Command (Save Without Committing)

```
git stash save "My work in progress" # stash changes
git stash list                       # list all stashes
git stash apply stash@{0}            # restore stash but keep it in list
git checkout -- .                    # discard applied stash changes
git stash pop                         # restore most recent stash and remove it
git stash drop stash@{0}             # delete specific stash
git stash clear                      # delete all stashes
```

Step 15 — Fork a Repository on GitHub

11. Go to any public GitHub repository (e.g. <https://github.com/other-user/project>)
12. Click the Fork button (top right)
13. GitHub creates a copy under your account
14. Clone your fork to work locally:

```
git clone https://github.com/YOUR_USERNAME/project.git
cd project
```

15. Make changes, commit and push to YOUR fork
16. To contribute back: click Pull Request on GitHub → compare your fork → click Create Pull Request

Command	Purpose
git branch name	Create a new local branch
git checkout name	Switch to a branch
git branch -a	List all local and remote branches
git merge name	Merge named branch into current branch
git rebase name	Rebase current branch on top of named branch
git branch -d name	Delete local branch (safe — only if merged)
git push origin --delete name	Delete branch from GitHub
git cherry-pick hash	Copy a commit to current branch

git reset --soft hash	Move commit back to staging area
git reset --hard hash	Discard commits and all changes
git revert hash	Create a new commit that undoes a commit
git reflog	Show all past HEAD positions for recovery
git stash save 'msg'	Save current changes without committing
git stash pop	Restore and remove most recent stash
git stash clear	Delete all stashes

Experiment No. 4 — CI/CD with Jenkins, Java & GitHub

Objective: Implement Continuous Integration/Continuous Deployment using Jenkins and Java.

Step 1 — Install Java JDK 11

17. Go to <https://adoptium.net> and download JDK 11 Windows x64 .msi
18. Run installer — check 'Set JAVA_HOME variable'
19. Verify:

```
java -version
javac -version
```

Step 2 — Install Git

20. Go to <https://git-scm.com/download/win> → download and install (keep all defaults)

```
git --version
```

Step 3 — Install Jenkins

21. Go to <https://www.jenkins.io/download/> → Windows → download .msi
22. Run installer → select 'Run service as LocalSystem' → keep port 8080
23. Open browser at <http://localhost:8080>
24. Get initial admin password:

```
type C:\Program Files\Jenkins\secrets\initialAdminPassword
```

25. Paste it into Jenkins → install suggested plugins → create admin user

Step 4 — Create Java Application

```
mkdir C:\HelloCI
cd C:\HelloCI
mkdir src\com\example
```

Create C:\HelloCI\src\com\example\HelloWorld.java:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World! CI/CD with Jenkins is working!");
    }
}
```

Create C:\HelloCI\build.bat:

```
@echo off
if not exist out mkdir out
javac -d out src\com\example\HelloWorld.java
jar cfe HelloWorld.jar com.example.HelloWorld -C out .
```

```
java -jar HelloWorld.jar
cd C:\HelloCI
.\build.bat
```

Step 5 — Push to GitHub

```
cd C:\HelloCI
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/HelloCI.git
git push -u origin main
```

Step 6 — Create Jenkins Job

26. Jenkins → New Item → HelloCI-Job → Freestyle project → OK
27. Source Code Management → Git → enter repo URL → Branch: */main
28. Build Triggers → Poll SCM → Schedule: *****
29. Build Steps → Execute Windows batch command:

```
if not exist out mkdir out
javac -d out src\com\example\HelloWorld.java
jar cfe HelloWorld.jar com.example.HelloWorld -C out .
java -jar HelloWorld.jar
```

30. Save → Build Now → click #1 → Console Output

Step 7 — Test CI Automatic Build

Change the Java file message, then:

```
git add .
git commit -m "Version 2"
git push
```

Jenkins detects the push within 1 minute and runs build #2 automatically.

Step 8 — Add Deploy Step (CD)

Update the Jenkins batch command to also deploy:

```
if not exist out mkdir out
javac -d out src\com\example\HelloWorld.java
jar cfe HelloWorld.jar com.example.HelloWorld -C out .
if not exist C:\Production mkdir C:\Production
copy HelloWorld.jar C:\Production\HelloWorld.jar
java -jar C:\Production\HelloWorld.jar
```

Error	Fix
-------	-----

build.bat not recognized in PowerShell	Use .\build.bat
Jenkins ERROR Couldn't find revision	Change */master to */main in job config
javac not found	Add JDK bin to System PATH
Git push asks for password	Use Personal Access Token as password

Experiment No. 5 — Docker Commands

Objective: Install Docker and implement common Docker commands for container management.

Step 1 — Install Docker Desktop on Windows

31. Go to <https://www.docker.com/products/docker-desktop/>
32. Download Docker Desktop for Windows
33. Run Docker Desktop Installer.exe as Administrator
34. During install: keep checked 'Use WSL 2 instead of Hyper-V'

Why WSL 2 over Hyper-V: WSL 2 is faster, uses less RAM, works on Windows Home (Hyper-V requires Pro only), and integrates better with Windows.

35. Restart PC when prompted
36. Wait for whale icon in taskbar to stop animating (Docker ready)

```
docker --version
```

Step 2 — docker pull

```
docker pull ubuntu:20.04
```

Step 3 — docker images

```
docker images
```

Step 4 — docker run

```
docker run --name mycontainer -it ubuntu:20.04 /bin/bash
# -it = interactive terminal    --name = give container a name
# Type commands inside container, then: exit
```

Step 5 — docker ps

```
docker ps          # show running containers
docker ps -a       # show ALL containers (including stopped)
```

Step 6 — docker start

```
docker start mycontainer
```


Step 7 — docker stop

```
docker stop mycontainer
```

Step 8 — docker rm

```
docker rm mycontainer      # remove stopped container
docker rm -f mycontainer   # force remove running container
```

Step 9 — docker rmi

```
docker rmi ubuntu:20.04
```

Step 10 — docker exec

```
docker run -d --name mycontainer ubuntu:20.04 sleep infinity
docker exec -it mycontainer /bin/bash
```

Step 11 — docker logs

```
docker logs mycontainer
```

Step 12 — docker inspect

```
docker inspect mycontainer
```

Step 13 — docker build

```
docker build -t myimage .
```

Step 14 — docker push

```
docker login
docker tag myimage YOUR_USERNAME/myimage:v1
docker push YOUR_USERNAME/myimage:v1
```

Step 15 — docker system prune

```
docker system prune  # cleans all unused containers, images, networks
```

Step 16 — Complete Lab Practice Sequence

```
docker pull ubuntu:20.04
docker images
docker run --name mycontainer -it ubuntu:20.04 /bin/bash
docker ps -a
docker start mycontainer
docker ps
docker stop mycontainer
docker rm mycontainer
docker rmi ubuntu:20.04
```

Command	Purpose	Example
docker pull	Download image	docker pull ubuntu:20.04
docker images	List images	docker images
docker run	Create+start container	docker run -it ubuntu /bin/bash
docker ps	List running	docker ps
docker ps -a	List all containers	docker ps -a
docker start	Start stopped	docker start mycontainer
docker stop	Stop running	docker stop mycontainer
docker rm	Remove container	docker rm mycontainer
docker rmi	Remove image	docker rmi ubuntu:20.04
docker exec	Enter container	docker exec -it c1 /bin/bash
docker logs	View output	docker logs mycontainer
docker build	Build from Dockerfile	docker build -t myimage .
docker system prune	Clean all unused	docker system prune

Experiment No. 6 — Docker Containerization (Java App)

Objective: Implement Docker containerization with a Java application.

Step 1 — Create Project Folder

```
mkdir C:\DockerLab
cd C:\DockerLab
```

Step 2 — Create HelloWorld.java

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World from Docker Container!");
        System.out.println("Containerization using Java is working!");
        System.out.println("Student: Omkar Barad | Experiment No: 6");
    }
}
```

Step 3 — Create Dockerfile (no extension)

```
# Use eclipse-temurin:11 (official replacement for deprecated openjdk:11)
FROM eclipse-temurin:11
WORKDIR /app
COPY HelloWorld.java /app/
RUN javac HelloWorld.java
CMD ["java", "HelloWorld"]
```

Dockerfile Line	Purpose
FROM eclipse-temurin:11	Official Java 11 image (openjdk:11 is deprecated and removed)
WORKDIR /app	Set /app as working directory inside container
COPY HelloWorld.java /app/	Copy your Java file from PC into container
RUN javac HelloWorld.java	Compile Java during image build
CMD ["java", "HelloWorld"]	Run the compiled program when container starts

Step 4 — Verify Folder Structure

```
ls C:\DockerLab # must show: HelloWorld.java Dockerfile
```

Step 5 — Build Docker Image

```
cd C:\DockerLab
docker build -t myjavaimage .
```

Step 6 — Verify Image Created

```
docker images
```

Step 7 — Run the Container

```
docker run --name myjavacontainer myjavaimage
```

Expected output:

```
Hello World from Docker Container!
Containerization using Java is working!
Student: Omkar Barad | Experiment No: 6
```

Step 8 — Verify Using docker logs

```
docker logs myjavacontainer
```

Step 9 — Check Status

```
docker ps -a    # Status: Exited (0) means success
```

Step 10 — Cleanup

```
docker rm myjavacontainer
docker rmi myjavaimage
```

Error	Fix
openjdk:11 not found	Use eclipse-temurin:11 (openjdk:11 was removed from Docker Hub in 2024)
Cannot connect to Docker daemon	Open Docker Desktop, wait for whale icon to stop
COPY failed file not found	Both files must be in C:\DockerLab
Conflict name already exists	Run docker rm myjavacontainer first

Experiment No. 7 — Maven Build Lifecycle

Objective: Demonstrate the Maven Build Lifecycle and its phases.

Step 1 — Install Maven on Windows

37. Go to <https://maven.apache.org/download.cgi> → Binary zip archive
38. Extract to C:\Maven\apache-maven-3.9.x
39. Set Environment Variables (Win+S → 'Environment Variables'):
 - System Variables → New → MAVEN_HOME = C:\Maven\apache-maven-3.9.x
 - System Variables → Path → Edit → New → C:\Maven\apache-maven-3.9.x\bin
40. Open a NEW Command Prompt and verify:

```
mvn -v
```

If 'mvn' is still not recognized — close ALL terminals and open a fresh new Command Prompt.

Step 2 — Create Maven Project Structure

```
mkdir C:\MavenDemo
cd C:\MavenDemo
mkdir src\main\java\com\example
```

Step 3 — Create HelloWorld.java

File: C:\MavenDemo\src\main\java\com\example\HelloWorld.java

```
package com.example;
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, Maven!");
        System.out.println("Student: Omkar Barad | Experiment No: 7");
    }
}
```

Step 4 — Create pom.xml

File: C:\MavenDemo\pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>MavenDemo</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>
  <properties>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
  </properties>
```

```
<build><plugins><plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-jar-plugin</artifactId>
  <version>3.3.0</version>
  <configuration><archive><manifest>
    <mainClass>com.example.HelloWorld</mainClass>
  </manifest></archive></configuration>
</plugin></plugins></build>
</project>
```

Step 5 — Run Each Maven Phase

```
cd C:\MavenDemo
```

Phase 1: mvn clean — Delete old build files

```
mvn clean
```

Phase 2: mvn validate — Check pom.xml is valid

```
mvn validate
```

Phase 3: mvn compile — Compile .java to .class

```
mvn compile
```

Phase 4: mvn test — Run unit tests

```
mvn test
```

Phase 5: mvn package — Create JAR file

```
mvn package
```

Phase 6: mvn install — Install JAR to local .m2 repo

```
mvn install
```

Step 6 — Run the Application

```
java -jar C:\MavenDemo\target\MavenDemo-1.0-SNAPSHOT.jar
# Output: Hello, Maven!
```

Step 7 — Run All Phases in One Command

```
mvn clean compile test package install
```

Phase	Command	What it does
Clean	mvn clean	Deletes target/ folder
Validate	mvn validate	Checks pom.xml is valid
Compile	mvn compile	Compiles .java -> .class
Test	mvn test	Runs unit tests

Package	mvn package	Creates .jar/.war file
Install	mvn install	Copies artifact to local .m2 repo
Deploy	mvn deploy	Uploads artifact to remote repo

Error	Fix
mvn not recognized	Restart CMD after setting PATH in System Variables
JAVA_HOME not set	Add JAVA_HOME = path to JDK in System Variables
Source option 5 not supported	Add maven.compiler.source=11 in pom.xml properties
JAR has no main manifest	Add maven-jar-plugin with mainClass in pom.xml

Experiment No. 8 — Selenium Automated Testing

Objective: Install Selenium and demonstrate automated web application testing.

Step 1 — Prerequisites

```
java -version
mvn -v
```

Step 2 — Install Google Chrome

41. Download from <https://www.google.com/chrome/>
42. Check version: Chrome menu → Help → About Google Chrome

Step 3 — Create Maven Project

```
mkdir C:\SeleniumLab
cd C:\SeleniumLab
mkdir src\test\java\com\example
mkdir src\main\java\com\example
```

Step 4 — Create pom.xml with Selenium Dependencies

File: C:\SeleniumLab\pom.xml — add these 3 dependencies:

```
<!-- Selenium WebDriver -->
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.18.1</version>
</dependency>
<!-- WebDriverManager: auto-downloads ChromeDriver, no manual setup needed -->
<dependency>
  <groupId>io.github.bonigarcia</groupId>
  <artifactId>webdrivermanager</artifactId>
  <version>5.7.0</version>
</dependency>
<!-- TestNG: run and report tests -->
<dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>7.9.0</version>
  <scope>test</scope>
</dependency>
```

Step 5 — SimpleTest.java

File: C:\SeleniumLab\src\test\java\com\example\SimpleTest.java

```
package com.example;
import io.github.bonigarcia.wdm.WebDriverManager;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.*;

public class SimpleTest {
    WebDriver driver;

    @BeforeTest
    public void setup() {
        WebDriverManager.chromedriver().setup();
        driver = new ChromeDriver();
        driver.manage().window().maximize();
    }

    @Test
    public void openWebsite() {
        driver.get("https://www.google.com");
        String title = driver.getTitle();
        System.out.println("Page Title: " + title);
        assert title.contains("Google");
        System.out.println("PASSED!");
    }

    @AfterTest
    public void teardown() { driver.quit(); }
}
```

Step 6 — LoginTest.java (Main Test)

Demo site: <https://www.saucedemo.com> | Username: standard_user | Password: secret_sauce

File: C:\SeleniumLab\src\test\java\com\example>LoginTest.java

```
package com.example;
import io.github.bonigarcia.wdm.WebDriverManager;
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.*;
import org.testng.Assert;
import org.testng.annotations.*;
import java.time.Duration;

public class LoginTest {
    WebDriver driver;  WebDriverWait wait;

    @BeforeTest
    public void setup() {
        WebDriverManager.chromedriver().setup();
        driver = new ChromeDriver();
    }
```

```

        driver.manage().window().maximize();
        wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    }

    @Test
    public void testSuccessfulLogin() {
        driver.get("https://www.saucedemo.com");
        driver.findElement(By.id("user-name")).sendKeys("standard_user");
        driver.findElement(By.id("password")).sendKeys("secret_sauce");
        driver.findElement(By.id("login-button")).click();
        wait.until(ExpectedConditions.urlContains("inventory"));
        Assert.assertTrue(driver.getCurrentUrl().contains("inventory"));
        System.out.println("LOGIN TEST PASSED!");
    }

    @Test
    public void testFailedLogin() {
        driver.get("https://www.saucedemo.com");
        driver.findElement(By.id("user-name")).sendKeys("wrong_user");
        driver.findElement(By.id("password")).sendKeys("wrong_pass");
        driver.findElement(By.id("login-button")).click();
        WebElement err =
wait.until(ExpectedConditions.visibilityOfElementLocated(
            By.cssSelector("[data-test='error']")));
        Assert.assertTrue(err.getText().contains("do not match"));
        System.out.println("FAILED LOGIN TEST PASSED!");
    }

    @AfterTest
    public void teardown() { driver.quit(); }
}

```

Step 7 — Run Tests

```

cd C:\SeleniumLab
mvn dependency:resolve
mvn test

```

Error	Fix
ChromeDriver not found	Install Google Chrome first
SessionNotCreatedException	WebDriverManager handles version mismatch automatically
NoSuchElementException	Use WebDriverWait before accessing elements
0 tests run	Test files must be in src\test\java\

Experiment No. 9 — TestNG Framework

Objective: Perform automated testing using the TestNG framework.

Step 1 — Prerequisites

```
java -version
mvn -v
```

Step 2 — Create Maven Project

```
mkdir C:\TestNGLab
cd C:\TestNGLab
mkdir src\test\java\com\example
```

Step 3 — Create pom.xml

Include: testng 7.9.0, selenium-java 4.18.1, webdrivermanager 5.7.0. Configure maven-surefire-plugin to use testng.xml.

Step 4 — Create testng.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name="TestNG Lab Suite" verbose="2">
  <test name="Calculator Tests">
    <classes><class name="com.example.CalculatorTest"/></classes>
  </test>
  <test name="String Tests">
    <classes><class name="com.example.StringTest"/></classes>
  </test>
  <test name="Annotation Tests">
    <classes><class name="com.example.AnnotationTest"/></classes>
  </test>
  <test name="Selenium Tests">
    <classes><class name="com.example.SeleniumTest"/></classes>
  </test>
</suite>
```

Step 5 — CalculatorTest.java

```
package com.example;
import org.testng.Assert;
import org.testng.annotations.Test;

public class CalculatorTest {
```

```
public int add(int a, int b)      { return a + b; }
public int subtract(int a, int b) { return a - b; }
public int multiply(int a, int b) { return a * b; }

@Test public void testAddition()      { Assert.assertEquals(add(10,5),
15); }
@Test public void testSubtraction()    {
Assert.assertEquals(subtract(10,5), 5); }
@Test public void testMultiplication() {
Assert.assertEquals(multiply(10,5), 50); }
@Test public void testDivision()      { Assert.assertEquals((double)10/5,
2.0); }
}
```

Step 6 — StringTest.java

```
package com.example;
import org.testng.Assert;
import org.testng.annotations.Test;

public class StringTest {
    @Test public void testLength()      { Assert.assertEquals("TestNG".length(),
6); }
    @Test public void testUpperCase() {
Assert.assertEquals("hello".toUpperCase(), "HELLO"); }
    @Test public void testContains()    { Assert.assertTrue("TestNG
Framework".contains("TestNG")); }
    @Test public void testTrim()        { Assert.assertEquals("  hi  ".trim(),
"hi"); }
}
```

Step 7 — AnnotationTest.java (All Annotations Demo)

```
package com.example;
import org.testng.Assert;
import org.testng.annotations.*;

public class AnnotationTest {
    @BeforeSuite public void beforeSuite() {
System.out.println("[BeforeSuite]"); }
    @BeforeClass public void beforeClass() {
System.out.println("[BeforeClass]"); }
    @BeforeTest public void beforeTest() {
System.out.println("[BeforeTest]"); }
    @BeforeMethod public void beforeMethod() {
System.out.println("[BeforeMethod]"); }
    @AfterMethod public void afterMethod() {
System.out.println("[AfterMethod]"); }
    @AfterClass public void afterClass() {
System.out.println("[AfterClass]"); }
    @AfterSuite public void afterSuite() {
System.out.println("[AfterSuite]"); }
```

```

@Test(priority=1) public void testFirst() { Assert.assertTrue(true); }
@Test(priority=2) public void testSecond() { Assert.assertTrue(true); }
@Test(enabled=false) public void skipped() { } // this test is skipped

@DataProvider(name="addData")
public Object[][] data() {
    return new Object[][] { {1,2,3},{5,5,10},{10,20,30}};
}
@Test(dataProvider="addData")
public void testAdd(int a, int b, int exp) { Assert.assertEquals(a+b, exp);
}
}

```

Step 8 — SeleniumTest.java (Selenium + TestNG)

Tests: testOpenBrowser (p1) → testSuccessfulLogin (p2, depends on p1) → testPageHeader (p3) → testProductsVisible (p4)

Step 9 — Run Tests

```

cd C:\TestNGLab
mvn dependency:resolve
mvn test

```

Step 10 — View HTML Reports

```
start C:\TestNGLab\target\surefire-reports\index.html
```

Annotation	When it runs	Use for
@BeforeSuite	Once before ALL tests	Global setup
@BeforeClass	Once before first test	Class setup
@BeforeTest	Before each <test> tag	Group setup
@BeforeMethod	Before EACH @Test	Open browser
@Test	The actual test	Test logic
@AfterMethod	After EACH @Test	Reset state
@AfterClass	Once after last test	Class cleanup
@AfterSuite	Once after ALL tests	Global cleanup

@DataProvider	Supplies test data	Data-driven tests
---------------	--------------------	-------------------

Experiment No. 10 — Full DevOps Lifecycle Web Application

Objective: Implement a web application using the complete DevOps lifecycle.

Tools: Git | GitHub | Maven | Jenkins | TestNG | Docker

Step 1 — Create Project Structure

```
mkdir C:\DevOpsApp
cd C:\DevOpsApp
mkdir src\main\java\com\example
mkdir src\main\webapp\WEB-INF
mkdir src\test\java\com\example
```

Step 2 — Create pom.xml

Packaging: war. Dependencies: javax.servlet-api 4.0.1, testng 7.9.0. Plugins: maven-war-plugin 3.4.0, maven-surefire-plugin 3.2.5 with testng.xml.

Step 3 — Create HelloServlet.java

File: C:\DevOpsApp\src\main\java\com\example\HelloServlet.java

```
package com.example;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
import java.io.*;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<h1>Hello from DevOps Pipeline!</h1>");
        out.println("<p>Git + GitHub + Maven + Jenkins + TestNG + Docker</p>");
        out.println("<p>Student: Omkar Barad | Experiment No: 10</p>");
    }
}
```

Step 4 — Create index.html

File: C:\DevOpsApp\src\main\webapp\index.html — simple HTML page with a link to /hello

Step 5 — Create web.xml

File: C:\DevOpsApp\src\main\webapp\WEB-INF\web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="4.0">
  <display-name>DevOps Web Application</display-name>
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
  </welcome-file-list>
</web-app>
```

Step 6 — Create AppTest.java

File: C:\DevOpsApp\src\test\java\com\example\AppTest.java — 6 TestNG tests:

- testApplicationStartup — basic sanity check
- testServletPath — verify /hello path
- testResponseContent — verify output text
- testAppVersion — verify version string
- testBusinessLogic — simple calculation
- testStringValidation — verify app name

Step 7 — Create testng.xml

File: C:\DevOpsApp\testng.xml — points to com.example.AppTest

Step 8 — Create Dockerfile

```
FROM tomcat:9.0-jdk11
RUN rm -rf /usr/local/tomcat/webapps/*
WORKDIR /usr/local/tomcat/webapps
COPY target/DevOpsApp-1.0-SNAPSHOT.war ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

Step 9 — Push to GitHub

```
cd C:\DevOpsApp
git init
git add .
git commit -m "Initial commit - DevOps web app Experiment 10"
git branch -M main
git remote add origin https://github.com/Omkarbarad/DevOpsApp.git
git push -u origin main
```

Step 10 — Configure Jenkins Pipeline

43. Jenkins → New Item → DevOpsApp-Pipeline → Freestyle project → OK

44. Source Code Management → Git → enter repo URL → Branch: */main

45. Build Triggers → Poll SCM → Schedule: * * * * *

46. Build Steps → Execute Windows batch command:

```
call mvn clean
call mvn compile
call mvn test
call mvn package
docker build -t devopsapp .
docker rm -f devopsapp-container 2>nul || echo No old container
docker run -d --name devopsapp-container -p 8085:8080 devopsapp
echo App running at http://localhost:8085
```

47. Save → Build Now

Step 11 — Verify the Application

- <http://localhost:8085> → shows index.html
- <http://localhost:8085/hello> → shows Hello from DevOps Pipeline!

Step 12 — Test CI/CD (Automatic Update)

Make a code change, then:

```
git add .
git commit -m "Version 2 update"
git push
```

Jenkins automatically detects → pulls → tests → builds WAR → creates Docker image → deploys new container. Refresh <http://localhost:8085> to see update.

Step	Tool	What Happens
1	Java + Maven	Web app code written with pom.xml
2	Git	Code committed locally with git commit
3	GitHub	Code pushed to remote with git push
4	Jenkins	Detects push via Poll SCM, triggers pipeline
5	Maven	Runs clean → compile → test → package
6	TestNG	6 tests run automatically inside Maven
7	Docker	WAR containerized and deployed on port 8085
8	Browser	App accessible at http://localhost:8085

Error	Fix
Port 8085 already in use	Run <code>docker rm -f devopsapp-container</code>
WAR not found in Dockerfile	Check mvn package output for errors

Tests failed BUILD FAILURE	Fix the failing test then re-push
Jenkins cannot find mvn	Add Maven to System PATH (Exp 7 steps)
docker not recognized in Jenkins	Restart Jenkins after installing Docker